

Artificial Intelligence

Section 4

By: Mohamed Fathy

Section Content

- Solving 8-puzzle Problem BFS

Breadth-first Search Pseudocode

function BREADTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure

node $\leftarrow$ a node with STATE = *problem*.INITIAL-STATE, PATH-COST = 0

if *problem*.GOAL-TEST(*node*.STATE) **then return** SOLUTION(*node*)

frontier $\leftarrow$ a FIFO queue with *node* as the only element

explored $\leftarrow$ an empty set

loop do

if EMPTY?(*frontier*) **then return** failure

node $\leftarrow$ POP(*frontier*) /* chooses the shallowest node in *frontier* */

 add *node*.STATE to *explored*

for each *action* **in** *problem*.ACTIONS(*node*.STATE) **do**

child $\leftarrow$ CHILD-NODE(*problem*, *node*, *action*)

if *child*.STATE is not in *explored* or *frontier* **then**

if *problem*.GOAL-TEST(*child*.STATE) **then return** SOLUTION(*child*)

frontier $\leftarrow$ INSERT(*child*, *frontier*)

Solving 8-puzzle
Problem

8-puzzle Problem

Start
State

1		2
4	5	3
7	8	6

Goal
State

1	2	3
4	5	6
7	8	

**Solving 8-puzzle
Problem**

- **Search Space:**

A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.

- **Start State:** Any state can be designated as the initial state. For example, In figure

- **Goal test:** This checks whether the state matches the goal configuration. For example, In figure

- **Actions:**

{Top, Right, Left, Down}

- **Path Cost:**

1 per action

- **Solution:**

Sequence of actions that change the 8-puzzle configuration from the start state to goal state

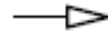
- **Optimal Solution: ?**

8-puzzle Problem

Go from state S to state G.

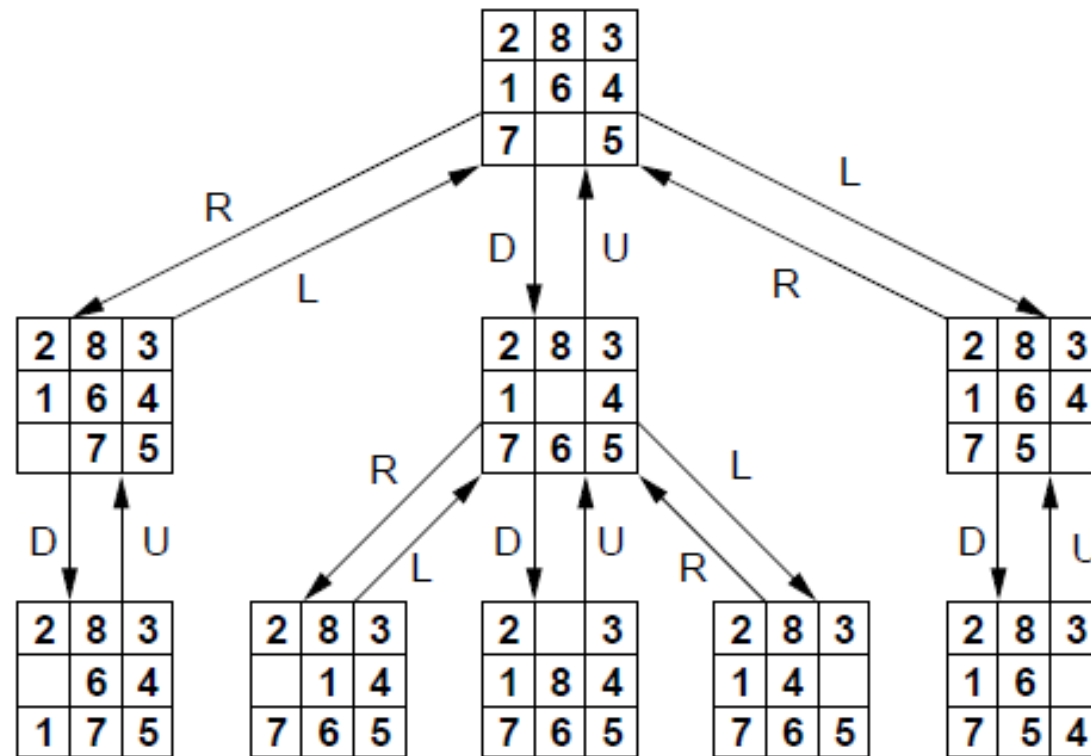
2	8	3
1	6	4
7		5

(S)



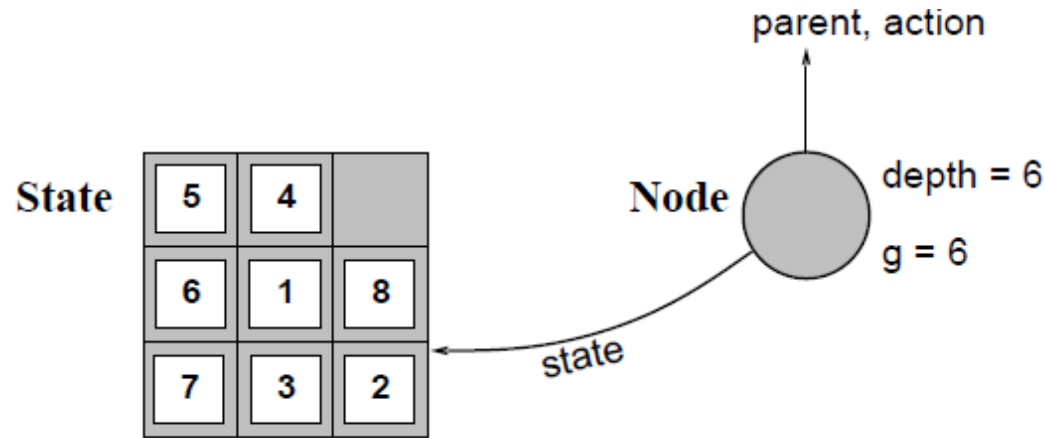
1	2	3
8		4
7	6	5

(G)



Solving 8-puzzle Problem

Implementation: states vs. nodes



Solving 8-puzzle Problem

- Up to now, we have not been very careful to distinguish between **nodes** and **states**, but in writing detailed algorithms it's important to make that distinction.
- A **state** corresponds to a configuration of the world.
- A **node** is a data structure constituting part of a search tree includes **parent**, **children**, **depth**, **path cost** $g(x)$
- **States** do not have parents, children, depth, or path cost.
- Furthermore, two different nodes can contain the same world state if that state is generated via two different search paths

8-Puzzle BFS Code

- **Time Complexity:** Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d = depth of shallowest solution and b is a node at every state.

$$T(b) = 1 + b^2 + b^3 + \dots + b^d = O(b^d)$$

- **Space Complexity:** Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.
- **Completeness:** BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.
- **Optimality:** BFS is optimal if path cost is 1 per step.

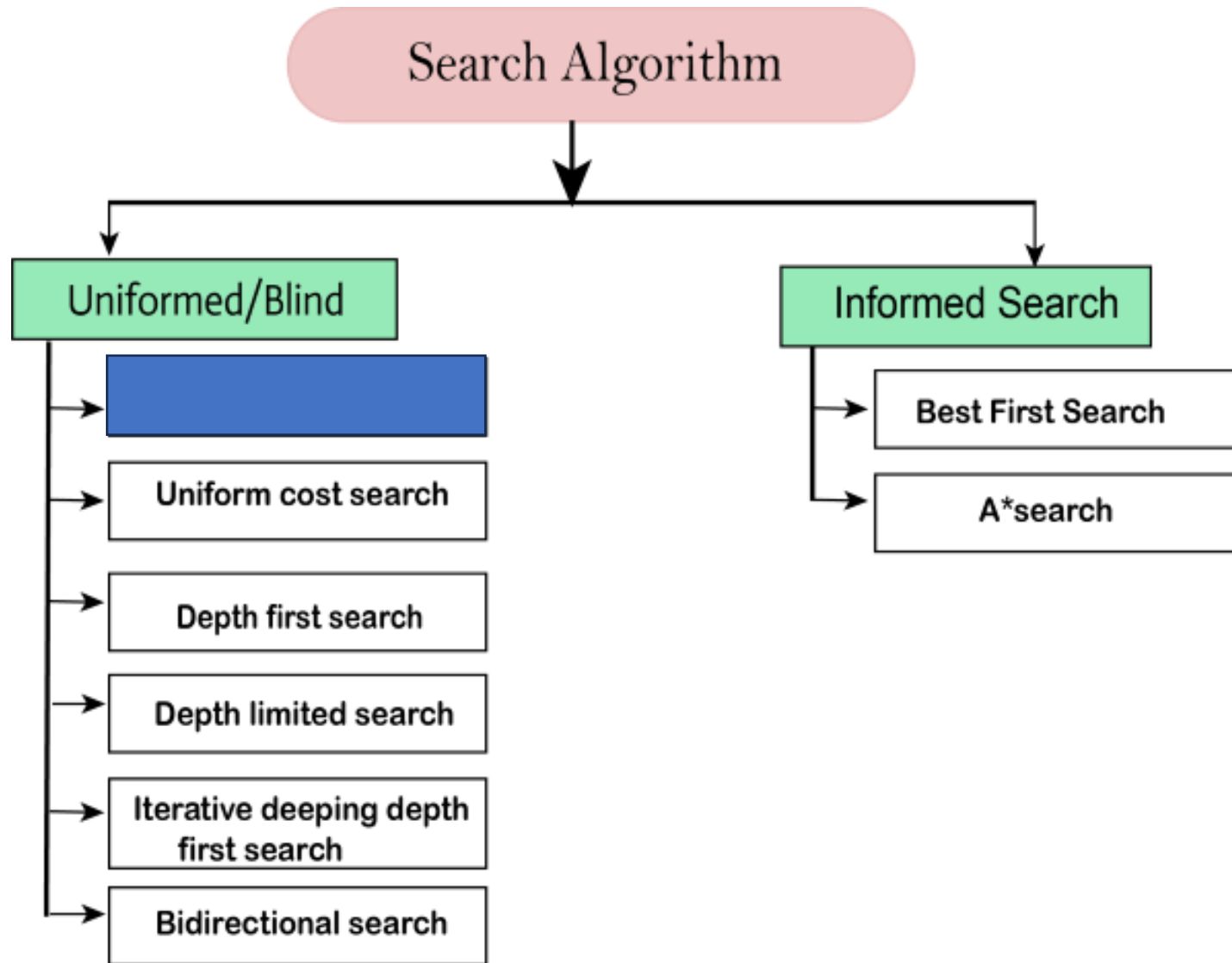
- Two lessons can be learned from Figure.

- First, the memory requirements are a bigger problem for breadth-first search than is the execution time.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

- One might wait 13 days for the solution to an important problem with search depth 12, but no personal computer has the petabyte of memory it would take. Fortunately, other strategies require less memory.

Solving 8-puzzle Problem



Solving 8-puzzle Problem